

SPECTRAL GRAPH THEORY AND THE MATRICES $J_n(\bar{j})$, $H_n(\bar{h})$: REMARKS ON THE DIGRAPH VIEWPOINT

These remarks concern the two matrices defined after Lemma 2.1 of *Oscillating spectra associated to certain sequences*, regarded as weighted adjacency matrices of directed graphs. The short answer is yes: spectral graph theory says quite a bit, and the directed-graph viewpoint fits these matrices unusually well. The remarks are organized from “structural” to “potentially useful for the μ_n project.”

1. THE DIGRAPH OF J_n HAS A VERY RIGID CYCLE STRUCTURE

Read $J_n(\bar{j})$ as the weighted adjacency matrix of a digraph on vertices $1, \dots, n$, with the convention that the entry A_{uv} is the weight of the edge $u \rightarrow v$. The superdiagonal gives a single “ascending” path $i \rightarrow i + 1$ with weight $-i$, and the lower-triangular Toeplitz part gives “descending” edges $i \rightarrow k$ ($k \leq i$) with weight j_{i-k+1} , including loops of weight j_1 . Because the *only* way to ascend is one step at a time, every directed cycle is supported on an interval $[k, m]$: it climbs $k \rightarrow k + 1 \rightarrow \dots \rightarrow m$ and returns via the single edge $m \rightarrow k$. Its weight is

$$(1) \quad w([k, m]) = j_{m-k+1} (-1)^{m-k} \frac{(m-1)!}{(k-1)!}.$$

2. THE COEFFICIENT THEOREM FOR DIGRAPHS REPROVES THEOREM 2.3 COMBINATORIALLY

The Cvetković–Sachs coefficient theorem, which is valid for weighted digraphs, says that the coefficient of x^{n-r} in $\chi(J_n)$ is

$$\sum_L (-1)^{c(L)} w(L),$$

the sum running over collections L of vertex-disjoint cycles covering r vertices, where $c(L)$ is the number of cycles in L and $w(L)$ is the product of the edge weights occurring in L . By the previous section, those collections are exactly sets of pairwise disjoint intervals in $\{1, \dots, n\}$ — that is, the coefficients are signed sums over partial composition-like structures, which is morally why partition and composition identities (MacDonald, Vein–Dale) appear at all in this setting.

This was verified numerically (see the script in Section 7): for a random integer sequence $\bar{j}$ with $j_1 = 1$ and $n = 7$, the cycle-cover sums match both

numpy 's characteristic polynomial and the formula of Theorem 2.3,

$$\binom{n}{r} r! h_r (-1)^r,$$

exactly.

A corollary worth noting: for digraphs the spectrum depends *only* on the multiset of cycle products. (Diagonal similarity is a “gauge transformation” that preserves cycle weights, and the coefficient theorem shows the characteristic polynomial is determined by them.) Hence the interval weights (1) are the complete intrinsic data behind the spectra under study. Any answer to the question “what is it about $\chi(J_n)$ that induces the observed oscillations?” must be expressible in terms of them.

3. STRONG CONNECTIVITY AND ITS STANDARD DIVIDENDS

The digraph is strongly connected (one can ascend stepwise and descend in one hop), so J_n is irreducible; and since the superdiagonal is nowhere zero, J_n is an unreduced Hessenberg matrix, hence nonderogatory — every eigenvalue has geometric multiplicity 1, and $\chi(J_n)$ coincides with the minimal polynomial.

Irreducibility upgrades the Geršgorin disc theorem via Taussky's theorem: an eigenvalue lying on the boundary of the union of the discs must lie on the boundary of *every* disc. More interestingly, Brualdi's cycle-based refinement of Geršgorin is spectral graph theory tailor-made for the present situation: it localizes eigenvalues using products of deleted row sums taken *around directed cycles* of the digraph, and here the cycles are exactly the intervals described above.

4. THE GERŠGORIN PICTURE EXPLAINS, GEOMETRICALLY, WHAT DEFORMATION DOES

Every diagonal entry of J_n equals j_1 , so all Geršgorin discs share one center. Deformation by c replaces every loop weight by c (and shifts the descending-edge weights), so it translates the common disc center to c while reorganizing the radii. In graph language, deformation is a uniform reweighting of the loops — the spectrum is pinned to circulate around c . This gives at least a heuristic for why the deformed spectra look more regular: the eigenvalues live in an annulus-like region about a fixed center, and μ_n measures the distance from 0 to a cloud orbiting c .

5. A CONCRETE LEVER FOR BOUNDING μ_n AWAY FROM ZERO

Since

$$\prod_i |\lambda_i| = |\det J_n| = n! |h_n|,$$

any upper bound ρ on the spectral radius coming from the graph (Geršgorin row sums in terms of the j 's, or Brualdi's cycle bounds) yields

$$\mu_n \geq \frac{n! |h_n|}{\rho^{n-1}}.$$

This is exactly the shape of statement the project wants — it reduces “the spectrum is zero-free” to controlling the top of the spectrum graph-theoretically, plus knowing $h_n \neq 0$. For the Lehmer application the logic unfortunately runs the wrong way ($h_n \neq 0$ is the desired conclusion, not a hypothesis). Still, for the deformed matrices, where only zero-freeness is needed, it is potentially usable.

6. CAVEATS, AND THE MATRIX H_n

One important caveat: the classical heavy machinery of spectral graph theory (Perron–Frobenius, eigenvalue interlacing, expander-type bounds) assumes nonnegative or symmetric matrices. The weights here are signed and the matrices are far from normal, so the spectra are genuinely complex point sets and those tools do not apply. What survives is exactly the digraph toolkit above: the coefficient theorem, irreducibility, and cycle-based inclusion regions.

Everything said for J_n transfers to $H_n(\bar{h})$ with the obvious changes: the superdiagonal weights are all $+1$, and the first-column edges $i \rightarrow 1$ carry weight $i h_i$. Consequently the cycles of the H_n digraph are the loops, the intervals $[k, m]$ with $k \geq 2$, and the intervals $[1, m]$, the last carrying the extra factor m .

7. THE PYTHON VERIFICATION SCRIPT

The following script (i) builds $J_n(\bar{j})$ for a random integer sequence with $j_1 = 1$; (ii) computes its characteristic polynomial with **numpy**; (iii) recomputes the coefficients from the digraph coefficient theorem, enumerating all sets of pairwise disjoint intervals and using the cycle weights (1); and (iv) compares both against the formula of Theorem 2.3, with h recovered from j via equation (D1). All three computations agree.

```
import numpy as np
import random
from math import comb, factorial

random.seed(1)
n = 7
# random integer j-sequence with j_1 = 1 (as in Lemma
# 2.2)
j = [None, 1] + [random.randint(-4, 4) for _ in range(2,
n + 1)] # j[1..n]
```

```

# Build J_n(jbar): lower part Toeplitz j_{i-k+1},
# superdiagonal -i
J = np.zeros((n, n))
for i in range(1, n + 1):
    for k in range(1, n + 1):
        if k <= i:
            J[i - 1, k - 1] = j[i - k + 1]
        elif k == i + 1:
            J[i - 1, k - 1] = -i

# Characteristic polynomial via numpy:
# coefficients of x^n + a_1 x^{n-1} + ...
cp_numeric = np.poly(J)

# --- digraph coefficient theorem ---
# Cycles of the digraph = intervals [k,m]:
# weight = j_{m-k+1} * (-1)^{(m-k)} * (m-1)!/(k-1)!
def cycle_weight(k, m):
    return j[m - k + 1] * ((-1) ** (m - k)) * factorial(m
        - 1) // factorial(k - 1)

# Coefficient of x^{n-r}: sum over sets of disjoint
# intervals
# covering r vertices, with sign (-1)^{#cycles} (det(xI
# - A) convention)
intervals = [(k, m) for k in range(1, n + 1) for m in
    range(k, n + 1)]

def disjoint_sets(ivs):
    """Enumerate all sets of pairwise disjoint intervals.
    """
    out = [[]]
    def rec(start, chosen):
        for idx in range(start, len(ivs)):
            iv = ivs[idx]
            if all(iv[0] > c[1] or iv[1] < c[0] for c in
                chosen):
                chosen.append(iv)
                out.append(list(chosen))
                rec(idx + 1, chosen)
                chosen.pop()
    rec(0, [])
    return out

coeffs = [0] * (n + 1) # coeffs[r] multiplies x^{n-r}
coeffs[0] = 1
for S in disjoint_sets(intervals):
    if not S:

```

```

        continue
    r = sum(m - k + 1 for k, m in S)
    w = 1
    for k, m in S:
        w *= cycle_weight(k, m)
    coeffs[r] += ((-1) ** len(S)) * w

print("numpy_char_poly_coeffs: ", np.round(cp_numeric,
6))
print("digraph_cycle-cover_coeffs:", coeffs)
print("match:", np.allclose(cp_numeric, coeffs))

# --- confirm these equal Theorem 2.3's binom(n,r) r! h_r
#      (-1)^r,
#      with h recovered from j via equation (D1) ---
h = [1]
for m in range(1, n + 1):
    s = sum(j[r] * h[m - r] for r in range(1, m + 1))
    h.append(s / m)
thm = [comb(n, r) * factorial(r) * h[r] * (-1) ** r for r
in range(n + 1)]
print("Theorem 2.3 coeffs: ", thm)

```

Output for the seeded run ($n = 7$):

```

numpy char poly coeffs:      [ 1. -7. -21. 385. -805.
-609. 6433. -8041.]
digraph cycle-cover coeffs: [1, -7, -21, 385, -805, -609,
6433, -8041]
match: True
Theorem 2.3 coeffs:         [1, -7.0, -21.0, 385.0,
-805.0, -609.0,
6433.0, -8041.0]

```